Dentacore Authentication Design: Sessions, Redis & 2FA

Nov 4, 2026 · @Nam

Summary

Dentacore keeps server-side Redis sessions (opaque session ID in a cookie) instead of access/refresh JWTs, adds idle + absolute timeouts, and requires TOTP 2FA for staff.

Area	Decision
Session model	Random opaque session ID in an HttpOnly cookie, verified against Redis on every request
Why not JWT	Instant revocation would need a per-request denylist lookup anyway; extra complexity for no gain on a single API
Redis topology	Two instances: sessions + all short-lived auth state (noeviction, AOF on, HA) and cache (allkeys-lru, no persistence)
Extension	Extend TTL on every request (GETEX or GET + EXPIRE); no throttled-extension trick
Timeouts	Idle 30 min (sliding), absolute 12 h (stored in session)
Expiry UX	Input-triggered extend, server-side autosave, re-login modal with request replay
Remember me	Option 1: longer session class, MFA required, 7 d idle / 30 d absolute, per-clinic toggle
2FA	TOTP, mandatory for staff, server-side MFA challenge (5 min TTL, 5 attempts), recovery codes
Storage	Activation token SHA-256 hashed in Postgres; TOTP secret AES-256-GCM encrypted in Postgres

Session model: Redis sessions, not JWTs

We keep the opaque session ID in a cookie, checked against Redis on every request. That gives instant admin revocation and session tracking.

Why not access/refresh JWTs

- Instant revocation with JWTs needs a denylist or per-user session version checked on every request. That is a per-request lookup again.
- JWTs add a refresh flow, token rotation, reuse detection and client token handling.
- Healthcare data needs revocation to take effect immediately (e.g. a staff member is let go), not when a 15-minute token expires.

If we ever move to JWTs, use the hybrid pattern

- Short-lived access JWTs (5–10 min), verified statelessly.
- Refresh tokens stored server-side as the session record (session list, per-device logout).
- Revocation deletes the refresh record; for instant cutoff, add the access token's `jti` to a denylist with TTL = remaining lifetime, broadcast to instances.

This only pays off when many independent services must verify identity without a central store. A single NestJS API doesn't need it.

Redis capacity and scaling

One Redis node handles 100k concurrent users comfortably; availability is the real concern. All figures below are estimates to confirm by load testing.

Measure	Estimate at 100k concurrent users
Auth lookups	10–20k/sec (one request per 5–10 s); ~50k/sec in a heavy burst
Single-node capacity	~100–200k simple GET/GETEX ops/sec
Memory	100k sessions × ~0.5 KB ≈ 50 MB
Network	~20 MB/s at 20k ops/sec

What matters at that scale

1. **High availability.** Primary + replica with failover (Sentinel or a managed service). AOF persistence so sessions survive restarts.
2. **Node client.** One shared client per app instance; enable auto-pipelining (`enableAutoPipelining` in ioredis).
3. **O(1) commands only.** No `KEYS *` or large values. Keep a per-user set of session IDs for listing sessions.
4. App servers and Postgres need horizontal scaling before Redis does.
5. Beyond one node, Redis Cluster scales roughly linearly; session keys shard cleanly.

Throttled-extension trick: not used

- The trick: only EXPIRE when remaining TTL < IDLE/2, pipelined GET + TTL otherwise.
- It saves write traffic (replication, AOF), not commands: 2 commands per request vs 1 for GETEX. Worse on per-command billing.
- It makes the idle timeout fuzzy (between half and the full value).
- Only worth it near single-node limits (~50k+ session ops/sec, sustained Redis CPU > 60–70%, or replication lag).

Verify: `redis-benchmark -t get,set -d 512 -c 100` on the target instance, then k6 against the NestJS auth guard.

Separate Redis instances: sessions vs cache

Sessions and cache need opposite eviction and persistence settings, and both are per instance, so they get separate instances before production.

	Sessions	Cache
When memory is full	Must never be evicted	Should be evicted
`maxmemory-policy`	`noeviction`	`allkeys-lru` / `allkeys-lfu`
Persistence	AOF on	Off
If it goes down	Everyone locked out	App slower, still works

Why one instance fails

- `allkeys-lru`: cache bursts evict active sessions, causing random logouts.
- `noeviction`: a full cache makes writes fail, so logins fail.

- `volatile-lru` doesn't help; sessions have TTLs too.
- Logical databases (`SELECT 1`) share memory and policy, aren't supported in Cluster, and `FLUSHALL` wipes both.

In NestJS: two clients (`REDIS_SESSION_URL`, `REDIS_CACHE_URL`) injected under different tokens; `@nestjs/cache-manager` points at the cache instance only. Sharing one instance is fine for local development only.

Sessions instance keyspace

All short-lived auth state lives in the sessions instance, not the cache: the cache's eviction could drop a pending activation mid-setup, the cache should be flushable at any time, and these keys hold password hashes and encrypted TOTP secrets. They share the sessions instance's needs, so no third instance.

Key	Contents	TTL
`session:{hash}`	Session payload	Idle timeout, capped at absolute
`user:{id}:sessions`	Set of the user's session hashes	None; remove stale entries when a session is deleted or found expired
`mfa:{id}`	Login MFA challenge	~5 min
`activation:{tokenHash}`	Pending activation setup	~15 min

With `noeviction`, Redis never cleans up for you: set every temporary key's TTL in the creating command (`SET activation:{tokenHash} <value> EX 900`), never SET then EXPIRE. Keys without a TTL accumulate until memory fills and logins fail.

Session security checklist

The opaque-ID cookie session is OWASP's recommended model; its security depends on the details below.

Required

- [ ] Session ID from a CSPRNG: `crypto.randomBytes(32).toString('base64url')` (256 bits)
- [ ] New session ID on login and on privilege change (role or password change)
- [ ] Logout deletes the session from Redis, not just the cookie

- [ ] Cookie: `HttpOnly`, `Secure`, `SameSite=Lax`, `Path=/`, named `__Host-sid`, no `Domain`
- [ ] Idle timeout (sliding) + absolute timeout (`absoluteExpiresAt` in the session)
- [ ] CSRF: verify `Origin` against an allowlist on POST/PUT/PATCH/DELETE (SameSite misses sibling subdomains)
- [ ] Per-user set `user:{id}:sessions` for listing and revoking sessions
- [ ] Revoke all sessions on password change/reset
- [ ] Update or revoke sessions when an admin changes a user's role
- [ ] HTTPS + HSTS; CORS `credentials: true` only for an explicit allowlist
- [ ] Never log session IDs; rate-limit login; Redis on a private network with ACL

Cheap and worth doing

- Store `session:{sha256(id)}`, not the raw ID, so a Redis leak yields no usable cookies.
- Store IP and user agent for display only (device list).
- Strict Content Security Policy against XSS.

Over-engineering (don't)

- Binding the session to an IP address (random logouts on mobile and clinic networks).
- User-agent or fingerprint checks.
- Rotating the session ID on every request (races between concurrent requests).
- Signing or encrypting a cookie that already holds a 256-bit random ID.
- Encrypting session payloads in Redis once keys are hashed.
- Stacking several CSRF defenses; SameSite + Origin check is enough for a JSON API.

Rule of thumb: if it breaks things for legitimate users, or you can't name the attack it stops, it's over-engineering. Higher-value next steps: MFA for staff and audit logging of patient-record access.

Timeouts: idle and absolute

Each session has two clocks: idle 30 min (sliding) and absolute 12 h (fixed); whichever runs out first ends it.

Clock	Behaviour	Protects against	Stored in
Idle	Pushed forward by every authenticated request	Walk-away on a shared clinic computer	Redis key TTL
Absolute	Never moves	Long-lived stolen cookies; forces periodic re-check of MFA, password, account status	`absoluteExpiresAt` in the session payload

Sliding-only is the common mistake: an active user or a polling tab keeps the session alive forever.

Per-request logic

```
session = GET session:{hash}
if !session → 401
remaining_absolute = absoluteExpiresAt - now
if remaining_absolute <= 0 → DEL, 401
EXPIRE session:{hash} min(IDLE_TTL, remaining_absolute)
```

Capping the TTL at the remaining absolute time makes Redis delete the key exactly at the hard limit.

Values: idle 30 min, absolute 12 h (one shift). Make them configurable per clinic later only if asked. A future patient portal can use more relaxed limits.

Session expiry UX

A user who types for longer than the idle timeout without any request gets a 401 on submit; we prevent that with input-triggered extension and autosave, and recover with a re-login modal that replays the request.

Why the 401 happens

1. The dentist opens the notes form: the last request the server sees.
2. Typing sends nothing to the backend.
3. At 30 min the Redis key expires and is deleted.

4. Submit at 40 min: `GET` returns nil → 401. Sliding expiry can't revive a deleted session, and the absolute timeout lives inside that deleted session, so nothing recreates it. Only a login creates a session.

Prevention

- **Input-triggered extension:** debounced extend request (at most every 5 min) fired by real input events, never by a timer.
- **Server-side autosave** of drafts every 1–2 min while content changes; doubles as the activity signal.
- **Expiry warning** ("logged out in 2 min — stay signed in?") for long reading without typing; expose remaining time via a response header.
- **Background requests don't extend:** polling, heartbeats and dashboards authenticate but skip sliding (e.g. a `@NoSlide()` decorator).
- **No `localStorage` / `sessionStorage` for drafts:** clinical notes on shared computers would be readable by the next user.
- Multiple tabs: share expiry state via `BroadcastChannel` so the warning shows in the active tab.

Recovery: re-login modal with replay

1. Submit returns 401.
2. Form state stays in memory (component / form library / store state); a login modal opens over the page. Never redirect to `/login` (it unmounts the form).
3. The modal shows the locked username ("Enter the password for dr.lan@clinic") with a "Sign in as someone else" link that does a full logout + reload.
4. After login, the frontend compares the new session's user ID with the one loaded at page start. Same user → replay; different user → discard queue and form state, reload.
5. The server attributes the replayed request to the new session's user and authorizes it as usual, so the frontend check prevents accidental misattribution but is not a security boundary.

Request queue (frontend, in memory)

Not RabbitMQ, and not a browser feature: browsers, `fetch` and axios don't retry 401s. Implement it in an axios response interceptor (~30–50 lines):

1. On 401, push `{ config, resolve, reject }` onto an array.
2. Open the modal once, however many requests fail together.
3. On same-user re-login, replay all and resolve the original promises.

4. On cancel, failure or different user, reject all and clear.

Details: exclude the login endpoint from the interceptor (wrong password is also 401); mark replayed requests (`_retried`) and reject on a second 401; never send `WWW-Authenticate` from the backend (triggers the browser's Basic-auth popup).

Remember me

We keep "remember me" and implement it as a longer session class (Option 1), not a separate remember-me token.

	Option 1: longer session class (chosen)	Option 2: separate remember-me token
How	Same Redis session, `remember: true` , longer TTLs, cookie with `Max-Age`	Long-lived hashed token in its own cookie; creates a fresh session when the normal one expires, then rotates
Revocation	Existing paths unchanged	Second credential to track and revoke
Risk	None extra	Concurrent tabs trip rotation reuse detection, causing false logouts

Without "remember me" the cookie has no `Max-Age` (browser-session cookie). Browsers may restore these, so server-side timeouts remain the real control.

Safeguards for staff

- Require MFA to create a remember-me session.
- Limits: 7 days idle, 30 days absolute.
- Checkbox label: "Keep me signed in — don't use on shared computers."
- Per-clinic setting to disable it.
- Mark remember-me sessions in the session list.

Related, lower-risk features: remember the username (prefill), and "remember this device for MFA" for 30 days (still requires the password and normal timeouts).

2FA onboarding: activation flow

An admin invites the user by email; the user sets a password and TOTP, and nothing is committed until the first OTP verifies.

Rules

- Activation token: `crypto.randomBytes(32)`, stored hashed, single-use, expires in 24–72 h. Re-inviting revokes earlier tokens.
- GET is read-only. Email scanners (Defender Safe Links, Mimecast, Proofpoint) and link previews open links automatically, some running JavaScript. Only a POST carrying human input (password, OTP) may consume the token. No auto-submit on page load.
- One TOTP secret per setup. After step 2 it exists in Redis (pending, encrypted) and in the user's authenticator app; no `TotpCredential` row exists yet.

Steps

1. **Open link: validate only.** `GET /auth/activation?token=…` hashes and looks up the token, checks expiry/used, returns `valid` + email to display. Changes nothing.
2. **Submit password: prepare.** `POST /auth/activation/setup { token, password }` validates token and password rules, generates the pending TOTP secret, and stores a pending setup in Redis keyed by `tokenHash` (TTL ~15 min): `{ tokenHash, passwordHash, encryptedSecret }`. Returns the QR / `otpauth://` URI. Token not consumed; restarting replaces the pending setup (tell the user to rescan and delete the old entry).
3. **Submit OTP: commit atomically.** `POST /auth/activation/complete { token, otp }` verifies the OTP against the pending secret, then in one transaction:
   - `UPDATE … SET used_at = now() WHERE token_hash = ? AND used_at IS NULL` (0 rows → abort)
   - save the password hash, create the `TotpCredential` row, save hashed recovery codes

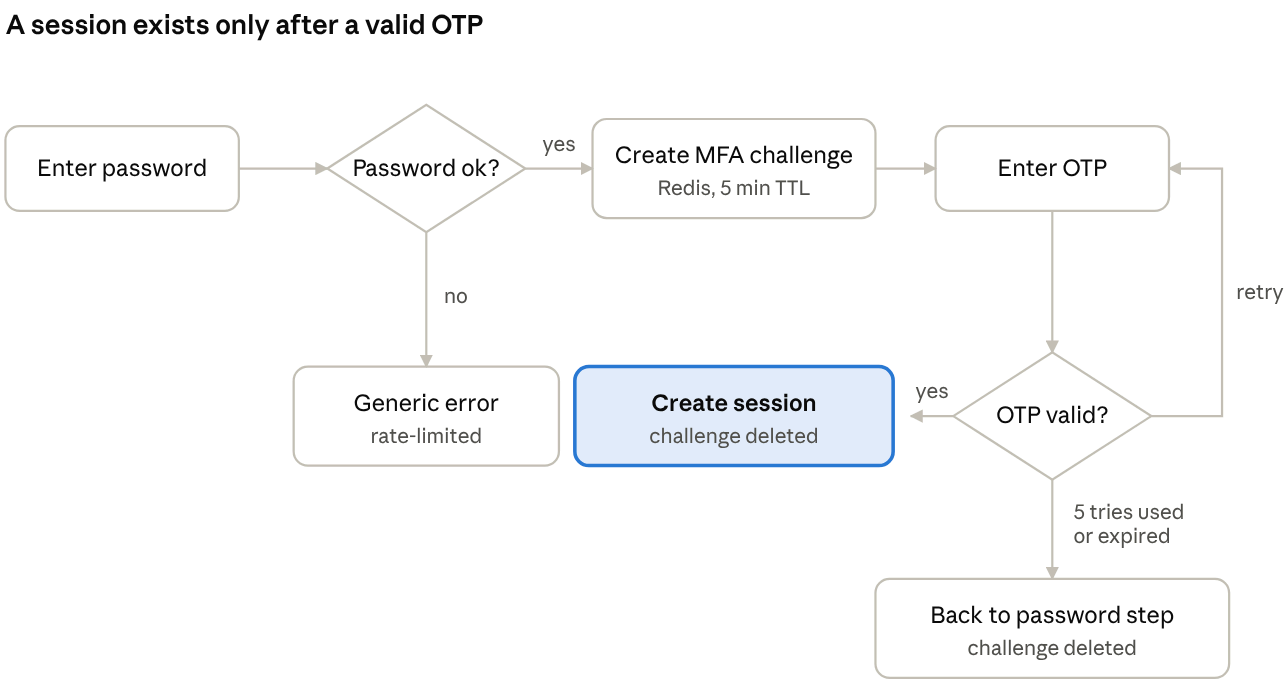
   Then delete the Redis pending setup (if that fails, its TTL cleans up), create a fresh session, show the recovery codes once.

Token in the URL

- Remove it from the address bar with `history.replaceState` after reading it.
- `Referrer-Policy: no-referrer` and no third-party scripts on the activation page.
- Don't log full URLs containing the token.

2FA login flow

A correct password creates only a short-lived MFA challenge in Redis; the real session is created after a valid OTP.



2FA login flow · 2 decisions, 1 retry loop

A wrong OTP retries within the same challenge; after 5 tries or 5 minutes the user starts again from the password.

Policy: 2FA is mandatory for staff, enforced as clinic policy rather than a personal setting. (Original idea: on by default, can be turned off in settings.) If a toggle is ever kept, turning it off requires password + current OTP and sends an email notification. Optional 2FA is reasonable for a future patient portal.

Server-side OTP timeout (replaces the hidden form timeout)

- A client-side hidden field or JS timer enforces nothing.
- Correct password → create `mfa:{randomId}` → `{ userId, attempts }` in Redis, TTL ~5 min; give the ID to the client in a short-lived cookie.
- OTP endpoint: challenge missing/expired → back to the password step; OTP valid → delete the challenge, create the session.

Attempt limits

- ~5 attempts per challenge, then delete it.

- Per-account backoff/lockout across challenges, so re-entering the password doesn't grant fresh attempts.

TOTP details

- Accept ±1 time step (30 s) for clock drift.
- Reject reused codes: store `lastUsedStep` per user.
- Use `otplib`; don't implement TOTP by hand.
- Recovery codes (about 10, single-use, shown once) cover a lost phone.

Storage: tokens, secrets, recovery codes

Activation tokens are SHA-256 hashed and TOTP secrets AES-256-GCM encrypted, both in Postgres (Prisma); recovery codes are argon2id hashed.

Item	Where	Protection	Why
Activation token	Email (raw), Postgres `ActivationToken.tokenHash`	SHA-256	256-bit random: no brute force; must be looked up by hash (salted bcrypt can't be)
TOTP secret	Redis (pending) → Postgres `TotpCredential.secretEnc`	AES-256-GCM	Server needs the original to compute codes
`TOTP_ENC_KEY`	Environment variable from the host's secret manager	Never in the DB	A DB leak must not include the key
Password hash	Redis (pending) → `User.passwordHash`	Password hash	Committed only at activation
Recovery codes	Postgres `RecoveryCode.codeHash`	argon2id	Shorter codes (~10 chars) need slow hashing

Schema

```
model User {
  id           String     @id @default(uuid())
  email        String     @unique
  passwordHash String?             // null until activation completes
  status       UserStatus @default(INVITED) // INVITED | ACTIVE | DISABLED
}

model ActivationToken {
  id          String    @id @default(uuid())
  userId      String
  tokenHash   String    @unique     // sha256(rawToken), hex
  expiresAt   DateTime
  usedAt      DateTime?
  revokedAt   DateTime?            // set on resend/cancel
  createdById String               // inviting admin/staff
  createdAt   DateTime  @default(now())
  user        User      @relation(fields: [userId], references: [id])
}

model TotpCredential {
  id           String   @id @default(uuid())
  userId       String   @unique
  secretEnc    String            // "v1:<iv>:<authTag>:<ciphertext>" base64
  lastUsedStep Int?              // replay protection
  activatedAt  DateTime
  createdAt    DateTime @default(now())
  user         User     @relation(fields: [userId], references: [id])
}

model RecoveryCode {
  id       String    @id @default(uuid())
  userId   String
  codeHash String             // argon2id
  usedAt   DateTime?
}
```

The user row is created at invite time (`INVITED`, no password), so the token always points at a real user.

Encryption rules

- Fresh random 12-byte IV for every encryption; never reuse with the same key.
- `v1:` prefix = key version, for rotation.

- User ID as AAD binds the ciphertext to its user; copying it to another row fails decryption.
- Decrypt: split, `createDecipheriv`, same `setAAD`, `setAuthTag`, `final()` (throws on tampering).

```
function encryptSecret(plain: string, userId: string): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', key, iv);
  cipher.setAAD(Buffer.from(userId));
  const ct = Buffer.concat([cipher.update(plain, 'utf8'), cipher.final()]);
  return ['v1', iv, cipher.getAuthTag(), ct].map((p, i) => i ?
p.toString('base64') : p).join(':');
}
```

Open items

- [ ] Design TOTP secret regeneration for a compromised secret. Starting points: during regeneration the active secret stays in Postgres while the new one is pending in Redis until verified, so the user can still log in; recovery codes cover the lost-device case.
- [ ] Load-test the session guard (k6) and Redis (`redis-benchmark`) on the target instance size.
- [ ] Decide the hosting secret manager for `TOTP_ENC_KEY`.